

Object-Oriented Programming (OOP) in Python

**Prepared by
Asif Farooq**

Introduction to OOP in Python

- ▶ OOP is a programming paradigm based on objects and classes.
- ▶ Works on DRY principle meaning “Don't Repeat Yourself”
- ▶ It allows code reusability, modularity, and organization.
- ▶ Python supports OOP with classes, objects, inheritance, polymorphism, encapsulation, and abstraction.

Key OOP Concepts

- ▶ Class: A blueprint for creating objects.
- ▶ Object: An instance of a class with attributes and behaviors.
- ▶ Class Attributes: Shared across all instances.
- ▶ Instance Attributes: Unique to each instance.
- ▶ Encapsulation: Hiding data for protection.
- ▶ Inheritance: Creating a new class from an existing class.
- ▶ Polymorphism: Same function, different behavior.
- ▶ Abstraction: Hiding complex details.

Class & Object

Name:_____
Class:_____
Roll No_____
Blood Group_____

Class



Name:Asif Farooq
Class:12
Roll No:56
Blood Group:A+

Object



Understanding Class and Object with Example

► Example:

```
class Car:
```

```
    brand = "Maruti"
```

```
    model = "XXL"
```

```
    year = 2010
```

```
    def display_info(self):
```

```
        print(f"Car Brand: {self.brand}, Model: {self.model},  
Year: {self.year}")
```

```
car1 = Car()
```

```
car1.display_info()
```

Understanding Class and Object with Example

► Example:

```
class Car:
```

```
    def __init__(self, brand, model, year):
```

```
        self.brand = brand
```

```
        self.model = model
```

```
        self.year = year
```

```
    def display_info(self):
```

```
        print(f"Car Brand: {self.brand}, Model: {self.model}, Year: {self.year}")
```

```
car1 = Car("Toyota", "Corolla", 2022)
```

```
car1.display_info()
```

Class vs. Instance Attributes

► Example:

```
class Company:
    company_name = "TechCorp"
    def __init__(self, employee_name):
        self.employee_name = employee_name
emp1 = Company("John")
print(emp1.company_name)
print(emp1.employee_name)
```

The `self` Parameter

- ▶ `self` represents the instance of the class.
- ▶ It is used to access instance attributes and methods.
- ▶ Example:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def introduce(self):
        print(f"Hello, my name is {self.name} and I am {self.age}
years old.")
p1 = Person("Asif", 30)
p1.introduce()
```


The `\_\_init\_\_` Constructor

- ▶ The `\_\_init\_\_` method is a constructor that initializes object attributes.
- ▶ Example:

```
class Student:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def show_details(self):
```

```
        print(f"Name: {self.name}, Age: {self.age}")
```

```
student1 = Student("John", 20)
```

```
student1.show_details()
```

Static Methods in Python

- ▶ Static methods do not depend on instance variables.
- ▶ Example:

```
class MathOperations:
```

```
    @staticmethod
```

```
    def add(a, b):
```

```
        return a + b
```

```
result = MathOperations.add(5, 10)
```

```
print("Addition Result:", result)
```

Summary

- ▶ OOP helps structure Python programs efficiently.
- ▶ Classes define blueprints for objects.
- ▶ Objects contain attributes and methods.
- ▶ `__init__` initializes attributes.
- ▶ Static methods belong to the class, not instances.
- ▶ `self` accesses instance attributes.

Programs

- ▶ Create a class “*Students*” for storing information of few students studying at Crescent Public School.
- ▶ Write a class “*Calculator*” capable of finding square, cube and square root of a number.
- ▶ Write a class “*Table*” that displays the table of a given number
- ▶ Create a class “*Test*” with a class attribute marks; create an object from it and set ‘marks’ directly using ‘object.marks =500’. Does this change the class attribute?
- ▶ Create a static method in Class Employee, to greet the user with “Welcome to Python”.
- ▶ Write a Class ‘Train’ which has methods to book a ticket, get status (no of seats) and get fare information of train running under Indian Railways.